Random Forests and Adaptive Boosting for the Classification of Network Attack Categories

Student number: 22677267

***Abstract*—Single classification trees are unstable. The attack categories of network flows are also imbalanced, and the flows include repeated feature vectors with conflicting labels. The study measured how much ensembles of classification trees improve the classification of attack categories in the UNSW-NB15 data set, and through what mechanism. A random forest and an adaptive boosting ensemble were built from scratch, with class weights or a class-balanced bootstrap to counter the imbalance. Nine research questions were answered under two protocols. The compatible protocol follows the second assignment, and under the grouped protocol no test vector has an identical training vector. The macro-averaged F1 score rose from 0.580 for a single tree to 0.634 for a class-weighted forest. About four fifths of the gain of the plain forest came from a reduction of the net variance, under the bias-variance definitions of the study. The grouped protocol kept the ranking of the models, and on the 15 grouped partitions the gains over the tree did not shrink, apart from a small decrease for the plain forest. Only the single grouped generalisation set showed smaller gains, for example a gain of $+0.041$ for the weighted forest against $+0.062$ on the compatible generalisation set, and the two sets hold different rows. Boosting scored lower than the weighted forest and placed nearly all of the instance weight on the conflicting vectors, a pattern that agrees with sensitivity to label noise but does not prove the cause. A dissimilar ensemble with nearest-neighbour members did not improve on the best forest.**

I. Introduction

A network intrusion detection system assigns every recorded network flow to the normal class or to an attack category. The report studies ensembles of classification trees for the attack category task on the UNSW-NB15 data set of Moustafa and Slay [16]. The Introduction sets out the context and the open questions, the goals and the approach, and the main findings with the outline of the report.

A. Context and Open Questions

The second assignment compared k-nearest neighbours with a classification tree on the same task. Both models classified the rare attack categories poorly, with F1 scores of only 0.103 on Backdoor and 0.115 on Analysis for the class-weighted tree of the second assignment, rebuilt for the study. A single tree is also unstable, because a small change in the training instances changes the splits. In all, 39.0 per cent of the flows repeat the feature vector of another flow, and 15.6 per cent of the flows belong to feature vectors with conflicting labels.

Breiman showed that the error of a forest depends on the strength of the trees and on the correlation between the trees, and reported that random forests are more robust to noise than adaptive boosting [4]. Three matters remain open for the present data. First, no deterministic classifier fits the minority labels of a conflicting vector, and boosting might spend the later rounds on those labels. Second, averaging leaves the bias of a tree unchanged, so variance reduction alone does not guarantee a better recall of the rare classes. Third, an identical training vector lets even a single tree answer correctly, so the gain of an ensemble may shrink when no test vector has such a twin.

B. Goals and Approach

The main goal is to measure how much an ensemble improves on a single tree, and through what mechanism. Further goals are to learn how the control parameters trade the strength of the members against their diversity and to test the out-of-bag error as a validation signal. The study also compares the ensembles with the two models of the second assignment and tests dissimilar members.

A random forest and an adaptive boosting ensemble were built from scratch on the classification tree of a library written for the study.[1] Nine research questions guided the experiments. The questions were fixed before the experiments were run. Two of the nine were added after the initial comparison, before the experiments that answer the two questions. The comparison used a compatible protocol that follows the second assignment and a grouped protocol that keeps identical vectors together. The main contribution is a decomposition of the gain of tree ensembles into variance reduction and bias reduction under both protocols.

C. Principal Findings and Outline

The forests raised the macro-averaged F1 score of the tree from 0.580 to 0.615 without a class treatment and to 0.634 with class weights. The gain came mainly from a reduction of the net variance. Boosting scored lower than the weighted forest and placed nearly all of the instance weight on rows with conflicting labels. The grouped protocol kept the ranking of the models, and on the grouped partitions only the gain of the plain forest over the tree shrank slightly. Only on the single grouped generalisation set were the gains of all four ensembles smaller, by an amount that differed between the ensembles. A dissimilar ensemble with nearest-neighbour members did not improve on the best forest.

Section II gives the background on ensemble learning and on the two models of the second assignment. Section III justifies the choice of ensembles and describes the data and the implementation. Section IV sets out the research questions and

[1] Source code: https://github.com/Noir-Cpu/cs741-assignment4

the experiments. Section V reports and discusses the results, and Section VI concludes. In short, the conflicting vectors and the rare classes leave the size and the source of the gain of tree ensembles open, and the study measures both.

II. Background

An ensemble helps only when the members differ, and the kind of difference decides whether the ensemble lowers variance or bias. The Background sets out the principles of ensemble learning, bagging and random forests, and adaptive boosting. The two comparison models of the second assignment and the treatment of class imbalance follow.

A. Principles of Ensemble Learning

An ensemble combines the predictions of several models, the members, into one prediction. The error of a model on unseen instances has three components: noise in the labels, bias and variance. Breiman defined bias and variance for a classifier as components of the test set error. Breiman also showed that unstable classifiers such as trees have low bias and high variance, and that combining several versions of such a classifier reduces the variance [3]. Under zero-one loss, the decomposition is an operational definition and not an additive identity: bias is defined against the main prediction, the class predicted most often over many training sets, and the net variance turns negative when variation between training sets corrects a wrong main prediction.

Members must differ for the combination to help. Breiman showed that the error of a forest depends on the strength of the trees and the correlation between the trees [4], so a mechanism that decorrelates the members also weakens each member. Kuncheva and Whitaker, in contrast, questioned whether measured diversity helps to build an ensemble [13]. The experiments therefore measured diversity and related the measure to the score, instead of assuming either view. Let $a_{i,j}$ denote the number of the B members that predict class j for instance i of an evaluated set $\mathcal{T}$. The disagreement Θ of equation (1) is the probability that two members drawn without replacement predict different classes, averaged over the instances.

$$\Theta = \frac{1}{|\mathcal{T}|} \sum_{i \in \mathcal{T}} \Big(1 - \sum_{j=1}^{m} \frac{a_{i,j}(a_{i,j} - 1)}{B(B-1)} \Big) \tag{1}$$

Members of different families need a combination rule. A hard vote counts the predicted classes, and a soft vote averages the class distributions of the members. Stacking, as Wolpert showed, trains a second-level model on member guesses for held-out data [25]. The guesses must be out-of-fold predictions, made by members fitted without the instance. Guesses on the training instances of a member would reward the member that memorises most, and the meta-learner would trust that member. A vote with equal weights helps when the members have comparable accuracy and make weakly correlated errors. A much weaker member pulls the average towards the errors of that member, so the vote can fall below the best member.

Algorithm 1 Construction of a random forest

Input: a labelled training set, the number of members B, the number of features φ tried per split, tree parameters
Output: a forest of B trees

1: **for each** member c from 1 to B **do**
2: Draw a bootstrap sample of the training set and record the multiplicities $\chi_{c,i}$
3: Grow a classification tree on the sample with the instance weights $\chi_{c,i}$
4: At every node, select the best split among φ features drawn at random
5: Stop growing at the depth limit or at the minimum leaf size
6: **end for**
7: Aggregate the leaf distributions of all trees via equation (2)

B. Bagging and Random Forests

Breiman introduced bagging, where each member trains on a bootstrap sample of n instances drawn with replacement from a training set of n instances [2]. An instance appears in a sample with the probability $1 - (1 - 1/n)^n \approx 0.632$. About 37 per cent of the instances are therefore out of bag for each member, and the out-of-bag instances allow an estimate of the generalisation error without a validation set. A random forest also chooses φ features at random for the split at each node, and Algorithm 1 summarises the construction. Breiman showed that the generalisation error of a forest converges as the number of trees grows [4]. Let $P_{c,j}(\mathbf{x})$ denote the proportion of class j in the leaf that member c reaches for the instance $\mathbf{x}$, and let B denote the number of members. Equation (2) gives the forest prediction over the m classes.

$$\hat{y}(\mathbf{x}) = \underset{j \in \{1,\ldots,m\}}{\arg\max} \; \frac{1}{B} \sum_{c=1}^{B} P_{c,j}(\mathbf{x}) \tag{2}$$

Let $\chi_{c,i}$ denote the number of times that instance i occurs in the sample of member c. The out-of-bag prediction $\tilde{y}(\mathbf{x}_i)$ applies equation (2) to the members with $\chi_{c,i} = 0$ only. Probst *et al.* reviewed the hyperparameters of the random forest, including the number of trees, the number of features per split, the minimum node size and the sampling scheme [18].

C. Adaptive Boosting

Freund and Schapire derived a boosting algorithm from an on-line learning model with a multiplicative weight-update rule [8]. Hastie *et al.* extended the algorithm directly to the multi-class case. The extension is called SAMME, an abbreviation for stagewise additive modelling using a multi-class exponential loss, and is equivalent to forward stagewise additive modelling under that loss [11]. Let $p_{c,i}$ denote the weight of instance i at round c, with $p_{1,i} = 1/n$. Let $\mu_c(\mathbf{x})$ denote the class that member c predicts for $\mathbf{x}$, and ι the

indicator function. Equation (3) gives the weighted error σ_c of member c.

$$\sigma_c = \sum_{i=1}^{n} p_{c,i}\, \iota\big(\mu_c(\mathbf{x}_i) \neq y_i\big) \tag{3}$$

Equation (4) gives the model weight ν_c, where $\varrho \in (0, 1]$ denotes the learning rate. The term $\ln(m-1)$ keeps the weight positive for every member with an error below $1 - 1/m$, the error of random guessing.

$$\nu_c = \varrho\Big(\ln \frac{1-\sigma_c}{\sigma_c} + \ln(m-1)\Big) \tag{4}$$

Equation (5) updates the instance weights, and a normalisation makes the weights sum to one. Equation (6) gives the prediction, and Algorithm 2 summarises the procedure.

$$p_{c+1,i} \propto p_{c,i} \exp\big(\nu_c\, \iota(\mu_c(\mathbf{x}_i) \neq y_i)\big) \tag{5}$$

$$\hat{y}(\mathbf{x}) = \operatorname*{arg\,max}_{j \in \{1,\ldots,m\}} \sum_{c=1}^{B} \nu_c\, \iota\big(\mu_c(\mathbf{x}) = j\big) \tag{6}$$

Boosting fits the training instances ever more closely, so label noise is a risk. Vectors with conflicting labels act as label noise for every deterministic classifier, so the robustness of forests to noise favours forests on the present data, whereas the bias of the rare classes favours boosting. Friedman developed gradient boosting, a method that fits regression trees as the additive components to the gradient of a loss [9].

D. Models of the Second Assignment

The k-nearest neighbours classifier assigns a query the class that dominates among the k closest training instances under the Manhattan or the Euclidean distance, so the features need comparable scales. Cover and Hart showed that the error rate of the nearest-neighbour rule approaches at most twice the Bayes error rate as the training set grows [7]. Let $\Omega_k(\mathbf{x})$ denote the indices of the k nearest training instances of $\mathbf{x}$, and y_i the label of instance i. Equation (7) gives the prediction. The vote weight w_i is one under uniform voting and the inverse distance under distance-weighted voting. The class weight is $b_j = (n/(m\, n_j))^{\alpha}$, where n_j is the number of training instances of class j and $\alpha \in [0, 1]$ the class-weight power.

$$\hat{y}(\mathbf{x}) = \operatorname*{arg\,max}_{j \in \{1,\ldots,m\}} b_j \sum_{i \in \Omega_k(\mathbf{x})} w_i\, \iota(y_i = j) \tag{7}$$

Algorithm 3 gives the prediction with the exact search of the study, a search that never stores the full matrix of distances.

A classification tree partitions the feature space with univariate threshold tests. Breiman *et al.* developed the classification and regression tree framework [5], and Quinlan described the induction of decision trees [19]. Let $\pi_{t,j}$ denote the weighted proportion of class j at node t, so that the Gini index is $g(t) = 1 - \sum_j \pi_{t,j}^2$ and the entropy is $H(t) = -\sum_j \pi_{t,j} \log_2 \pi_{t,j}$. A split s sends the shares η_L and η_R of the node weight to the children t_L and t_R. The tree grows by choosing the split with the largest impurity decrease of equation (8), for the impurity ϕ equal to g or H, until a node is pure or reaches the depth limit or the minimum leaf size.

$$\Delta(s, t) = \phi(t) - \eta_L\, \phi(t_L) - \eta_R\, \phi(t_R) \tag{8}$$

Instance weights and class weights enter the proportions $\pi_{t,j}$. Each test depends on the order of the values of one feature only, so a monotone transformation of a feature leaves the tree unchanged. Algorithm 4 summarises the induction on binned features, the base learner of every ensemble of the study.

E. Class Imbalance in Ensembles

The data have a class ratio of 534 to one between the largest and the smallest class. Ting showed that the initial instance weights decide whether tree induction yields minimum-error trees or trees of minimum high-cost error [22]. A tree ensemble can therefore multiply the weight of each instance by the class weight b_j. Alternatively, the ensemble can change the sampling of each member, as a balanced bootstrap does. Liu *et al.* proposed exploratory undersampling, where a learner trains on each of several subsets of the majority class and the outputs are combined [15].

Algorithm 2 Multi-class adaptive boosting

Input: a labelled training set, the number of rounds B, the learning rate ϱ, the weak-learner parameters
Output: members μ_c with model weights ν_c

1: Set every instance weight $p_{1,i}$ to $1/n$
2: **for each** round c from 1 to B **do**
3: Grow a weak classification tree on the training set with the weights $p_{c,i}$
4: Compute the weighted error σ_c via equation (3)
5: Stop if σ_c reaches $1 - 1/m$
6: Compute the model weight ν_c via equation (4)
7: Update and normalise the instance weights via equation (5)
8: **end for**
9: Predict the class with the largest summed model weight via equation (6)

Algorithm 3 Prediction of k-nearest neighbours

Input: training instances $\mathbf{x}_i$ with labels y_i, a query $\mathbf{x}$, k, the distance, the vote and class weights
Output: the predicted class $\hat{y}(\mathbf{x})$

1: Fill a sorted list of k distances with infinite entries
2: **for each** training instance i, in tiles of 1,024 instances **do**
3: Accumulate the distance to $\mathbf{x}$ over blocks of 16 features and skip i once the sum reaches the k-th listed distance
4: Otherwise insert i into the list and drop the last entry
5: **end for**
6: Return the class with the largest vote of the listed instances $\Omega_k(\mathbf{x})$ via equation (7)

F. Summary

Bagging and random forests reduce variance, adaptive boosting reduces bias, and class weights or balanced sampling counter the class imbalance. The comparison models are k-nearest neighbours and the classification tree, and stacking and voting combine members of different families. The next section justifies the choice of ensembles.

III. Methodology

The design must separate the effect of ensembling from the effects of the base learner and the preprocessing, and must stay valid on repeated vectors. The Methodology justifies the choice of ensembles, describes the data and their transformation, and sets out the design of the ensembles. The implementation closes the section.

A. Choice of Ensemble Methods

The final evaluation is meant to show how much error a collection of trees removes compared with one tree, and through what mechanism. An ensemble of similar members keeps the base learner, the impurity measure and the preprocessing fixed. The comparison with the single tree therefore isolates the effect of ensembling better than a comparison with a model of another family. A heterogeneous ensemble would instead test whether another inductive bias corrects the errors that similar trees share, such as the memorisation of repeated vectors. A nearest-neighbour model shares that tendency, so the choice was also tested: votes and a stacked ensemble of the forests, the tree and the weighted nearest-neighbour model were compared with the best forest.

Two homogeneous methods cover the two mechanisms of Section II-A. A random forest of deep trees, with low bias and high variance, suits averaging, and adaptive boosting with trees of limited depth reduces bias. The library of the study provides only classification trees, so the study used the weighted variant of SAMME instead of gradient boosting. Both methods have class-sensitive variants, because the class ratio makes the unweighted variants favour the frequent classes. The plain forest uses neither class weights nor class-balanced sampling. The weighted forest uses class weights with the standard bootstrap, and the sampled forest uses a class-balanced bootstrap or undersampling.

Algorithm 4 Induction of a weighted classification tree

Input: binned weighted instances, the impurity ϕ, the depth limit, the minimum leaf size, φ
Output: a tree with weighted class totals in every leaf

```
Open the root node with every instance of positive weight
for each open node t do
    Store the weighted class totals of t
    if t is pure, at the depth limit or below twice the leaf size then
        Close t as a leaf
    else
        for each feature in random order until φ non-constant features are tried do
            Compute Δ(s, t) via equation (8) at every bin boundary that leaves enough instances on both sides
        end for
        Split t at the best boundary into t_L and t_R, or close t as a leaf if no boundary qualifies
    end if
end for
```

B. Data Set and Transformation

Moustafa and Slay created UNSW-NB15 from real background traffic and synthetic attack traffic [16]. The file holds 257,673 flows with an identifier, 39 numeric features, three nominal features and a target with ten classes: the normal class and nine attack categories. The largest class holds 93,000 flows and the smallest class, Worms, holds 174 flows. Always predicting the largest class reaches an accuracy of 36.1 per cent but a macro-averaged F1 score of only 0.053. The 15.6 per cent of the flows with conflicting labels belong to 2,275 distinct vectors. Predicting the most frequent label of every distinct vector reaches 91.6 per cent accuracy on the development set D itself, an upper bound for deterministic classifiers on those rows only. On average, 44.1 per cent of the test flows of a comparison partition have an identical twin in the training part, so cross-validated scores are optimistic for models that memorise.

A split depends only on the order of the values of one feature, so the logarithm and the standardisation of the distance-based pipeline leave every tree unchanged. An ablation on the single tree confirmed the argument, with a macro-averaged F1 score of 0.580 for the raw columns and 0.580 after the transformations. The ensembles therefore used the raw numeric columns and kept outliers and correlated columns. The identifier numbers the rows and describes no property of a flow, so the identifier was dropped. The missing symbol `?` occurs only in the feature service, in 54.8 per cent of the flows, and the share depends on the protocol and the connection state, so the symbol was kept as the category none. Protocols with fewer than 150 training instances, counted on the training part only, were merged into the category other. The threshold follows the pipeline of the second assignment. In the development set D, 118 of the 133 protocols fall below the threshold but hold only 5.9 per cent of the flows, and without the merge the one-hot encoding of the nearest-neighbour pipeline would grow from 74 to 191 columns. Strobl *et al.* showed that variable selection in random forests is biased when the predictors vary in the number of categories [21], and a one-hot indicator offers a single threshold. The study therefore compared the one-hot encoding of the three nominal features with an integer encoding by the frequency rank of each category in the training part, and kept the better encoding.

C. Design of the Ensembles

The experiments varied the number of trees, the number of features per split, the depth limit, the minimum leaf size and the sampling of the forest. The random forest extends

Algorithm 1 in three ways. First, a sampler controls the class balance of each sample, as Algorithm 5 summarises. Instance i is drawn with a probability proportional to $n_{y_i}^{-\omega}$, where $\omega \in [0, 1]$ is the class-balance exponent. The value $\omega = 0$ gives the standard bootstrap and $\omega = 1$ gives equal expected counts for every class. Undersampling instead draws q instances with replacement from every class. Second, the class-weight power α scales the weight of every instance in the impurity computation and the leaf distributions of the member trees. Third, the out-of-bag and the test predictions are stored at selected numbers of members while the trees are fitted, so one fit gives the scores and the diversity of every prefix of the forest. The boosting ensemble follows Algorithm 2, with initial instance weights proportional to the class weights. The weak learners are trees with a depth limit and all features at every split, so the fit is deterministic.

D. Implementation

A library written from scratch for the study supplies the classification tree, the nearest-neighbour model, the preprocessing, the partitions of both protocols, the seed streams and the statistical tests. The tree is implemented in NumPy [10] and Numba [14], and SciPy [24] supplies the statistical tests. Table I lists the main modules and their tests. Scikit-learn appears only in the unit tests as a reference, and all unit tests pass. The tests guarantee three properties that the results rely on: the split search finds hand-computed splits, a forest is identical for any number of threads, and the neighbour search returns the neighbours of a full distance computation.

All experiments ran on one machine with an AMD Ryzen AI 9 HX 370 processor with 24 logical cores and 27 gigabytes of memory. The fit kernel of the tree releases the global interpreter lock, so a pool of ten threads fits the trees of a forest in parallel. The trees return in index order, so a forest is reproducible for any number of threads. The trees use at most 4,095 bins per feature, with edges fitted on the training rows. A feature with at most 4,095 distinct values receives an edge midway between every pair of adjacent values, so the split search stays exact, and a feature with more values receives edges at equal-frequency quantiles. A one-hot indicator has two values and therefore one edge. In the ablation, a single tree with 4,095 bins reached 0.582 against 0.580 for the exact search.

The two models of the second assignment were rebuilt with the library. Distances need comparable scales, so the nearest-neighbour model standardises every numeric column and encodes the nominal features as one-hot indicators. The model also drops three columns that correlate at 0.995 or more with a retained column, because a near-copy counts the same quantity twice in the distance. The model also takes the logarithm of one plus the value of 25 heavy-tailed features, whose skewness of 1.64 to 78.8 fell to at most 5.38, so that a few extreme flows do not dominate the distance. In the ablation, the model without the logarithm scored 0.490 against 0.520. The search is exact brute force: blocks of 16 queries run in parallel, and the training matrix is streamed in tiles, as Algorithm 3 shows. Grids of 160 and 168 configurations, selected on the macro-averaged F1 score, tuned the neighbourhood size, the distance, the vote weighting and the class-weight power of the nearest-neighbour model, and the criterion, the depth limit, the leaf size and the class weights of the tree. For the plain nearest-neighbour model, the search selected five neighbours with the Manhattan distance and distance-weighted votes. For the weighted model, the search selected 15 neighbours with the class-weight power 0.25. For the plain tree, the search chose the entropy criterion and depth 25, and for the balanced tree depth 15 with balanced class weights. The rebuilt models stay close to the second assignment: the balanced tree scored 0.569 against 0.576, and the nearest-neighbour model 0.523 against 0.525.

Algorithm 5 Class-balanced sample of one tree

Input: labels y_i, class sizes n_j, the exponent ω, an undersampling size q or none
Output: the multiplicities $\chi_{c,i}$ of the sample of member c

if an undersampling size q is given **then**
  for each class j **do**
    Draw q instances of class j with replacement and count the repetitions
  end for
else
  Set the draw probability of instance i proportional to $n_{y_i}^{-\omega}$
  Draw n instances with replacement and count the repetitions
end if
Mark every instance with multiplicity zero as out of bag for the member

TABLE I: Modules and tests

Module	Responsibility	Test
data, preprocess	loading, transformations fitted on training rows	hand-computed preset, statistics from training rows only
protocol, grouped	seed streams, both kinds of partition	reproducible streams, disjoint partitions, groups kept whole
metrics, stats	measures, paired tests, Holm adjustment	hand examples, agreement with scikit-learn and SciPy
knn	exact search and weighted votes	hand-computed votes, agreement with brute force
tree	binned classification tree	hand-computed splits, parallel fits equal to serial fits
ensemble	forest, samplers, boosting, diversity	hand-computed bootstrap and boosting round, independence of the thread count
scripts	experiments, macros, figures	no unit test, numbers regenerated from the result files

E. Summary

The study built a random forest and a boosting ensemble from trees, with class-sensitive sampling and weights. The comparison with a single tree then measures the effect of ensembling with the same base learner. The next section states the research questions and the experiments.

IV. Empirical Procedure

Tuning gives insight only when the questions and expectations come before the results. The Empirical Procedure sets out the research questions with their expectations and analyses, the evaluation protocols and the experiments. The measures, the statistical tests and the changes to the initial design follow. The questions were fixed before the experiments were run. Seven questions were fixed before any experiment. The grouped and the dissimilar questions were added after the initial comparison, and each of the two was fixed with an expectation before the experiments that answer the question.

A. Research Questions

The size question asked how the number of members changes the error of the forest and of the boosting ensemble, and where the returns diminish. The forest was expected to level off near 100 trees without overfitting [4], and boosting to reach the optimum earlier and to degrade under label noise. The feature question asked how the number φ of features tried per split trades the strength of a tree against the diversity of the forest. Few features were expected to give weak and diverse trees. The structure question asked how the depth limit and the minimum leaf size affect the forest. Deep trees were expected to suit averaging, because averaging removes variance and not bias.

The treatment question asked whether class weights, a balanced bootstrap or undersampling help the rare classes, and at what cost, judged by the per-class recall, the balanced accuracy and the accuracy. A higher recall on the rare classes was expected, with lower precision and accuracy. The decomposition question asked how much of the gain over a single tree comes from reducing variance and how much from reducing bias, overall and for the rare classes. The forest was expected to reduce the variance and boosting the bias, with more variance on noisy labels. The share of the boosting weight on the rows of conflicting vectors was expected to rise with the number of rounds.

The out-of-bag question asked whether the out-of-bag error ranks configurations reliably and estimates the test error. The rank correlation between the out-of-bag and the test scores measures the ranking, and the agreement of the out-of-bag and the cross-validated choice measures the use for selection. The comparison question asked how the tuned ensembles compare with the models of the second assignment, and how repeated vectors affect the comparison, through paired tests under the compatible protocol. A twin is a training instance with the feature vector of a test instance, and a test instance has no twin, twins of one label, or twins of several labels. The gain was expected to be larger on the instances without a twin, because a twin already lets a single tree answer correctly.

The grouped question asked whether the ranking of the models and the conclusions hold when no test vector has a twin. The memorising nearest-neighbour models and fully grown trees were expected to lose most, and the ensembles to stay ranked above the single models. The answer rests on the same paired tests and on the Kendall rank correlation of the mean scores under the two protocols. The dissimilar question asked whether a vote or a stacked combination of the forests, the tree and the weighted nearest-neighbour model improves on the best forest, and what each member contributes through leave-one-member-out votes. No combination was expected to gain more than 0.005, because the nearest-neighbour model had scored 0.08 lower than the forest in the first results and shares the tendency to memorise.

B. Evaluation Protocols

A master seed, 1072417991, generated every random stream through the seed sequence of NumPy [10] and a named purpose. Compared configurations share the streams, so their differences are not seed noise. The compatible protocol follows the second assignment. A stratified hold-out fixed the generalisation set G of 51,534 flows, never used for tuning or selection, and left the development set D of 206,139 flows. Tuning used stratified five-fold cross-validation inside D. The comparison repeated stratified five-fold cross-validation three times and gave 15 paired scores per model on overlapping partitions. Preprocessing was refitted on every training part.

In the grouped protocol, two rows belong to the same group when all 42 descriptive features are identical. A grouped hold-out fixed D_g and G_g, and grouped stratified five-fold cross-validation, repeated three times, gave 15 partitions where no group straddles the training and the test part. The tuned configurations stayed unchanged, so the grouped scores measure the effect of the leakage and not a second tuning. Stochastic models ran with independent streams: two runs in the sweeps, three in the second stage of the joint search, five per comparison partition, and 30 and 10 runs on G and G_g. Boosting and the baselines are deterministic and ran once.

C. Experiments and Control Parameters

Table II lists the experiments. The base forest of the sweeps grows 200 trees with the entropy criterion of the tuned baseline tree. The base forest tries $\varphi = 8$ of the 79 encoded features per split, the rounded-down square root, and uses no depth limit, a leaf size of one, the standard bootstrap and $\alpha = 0$. The ordinal rank would replace the one-hot encoding only if the rank raised the mean macro-averaged F1 score by more than 0.005, half the standard deviation of the baseline tree over the comparison partitions. The returns diminish at the smallest number of members whose mean score lies within the same margin of the final score. The treatment samplers were the standard bootstrap, balanced bootstraps with the exponents ω of 0.5 and one, and undersampling to the smallest class and to 1,000 instances per class.

TABLE II: Experiments and control parameters

Experiment	Varied parameters and values	Fixed settings	Units of work
Encoding	one-hot against ordinal rank	base forest	five folds, two runs
Size	tuned forests to 2,000 trees, boosting to 1,000 rounds, 21 and 19 checkpoints	selected configurations	five folds, two runs
Feature	$\varphi \in \{1, 2, 4, \text{square root}, 16, 32, \text{all}\}$	base forest	five folds, two runs
Structure	depth limit $\{3, 5, 8, 12, 20, \text{none}\}$, leaf size $\{1, 2, 5, 10, 25, 50\}$	base forest	five folds, two runs
Treatment	$\alpha \in \{0, 0.5, 1\}$, five samplers	base forest	five folds, two runs
Joint search	criterion, φ, depth limit, leaf size, α, sampler; four families of class treatment	100 and 200 trees	stage one: two folds; stage two: five folds, three runs
Boosting	depth of the weak learner $\{3, 6, 9, 12, 18, 24, 32\}$; $\varrho \in \{0.03, 0.1, 0.3, 1\}$; $\alpha \in \{0, 0.5, 1\}$	200 and 300 rounds	five folds
Decomposition	tuned models, bagging and baseline trees	20 training sets of half the training part	20 fits per model, five folds
Boosting weight	share of the weight on conflicting rows after selected rounds	tuned boosting	five folds
Compatible comparison	tuned models, baselines and dissimilar ensembles	500 trees	15 partitions, five runs, G with 30 runs
Grouped comparison	the same models	500 trees	15 grouped partitions, five runs, G_g with 10 runs

The one-factor sweeps do not give the joint optimum, so a joint search covered the criterion, $\varphi \in \{4, 8, 12, 16, 24, 32, 48\}$, the depth limit $\{16, 20, 24, 32, \text{none}\}$, the minimum leaf size $\{1, 2, 3, 5\}$ and $\alpha \in \{0, 0.25, 0.5, 0.75, 1\}$. The sampler was the standard bootstrap, a balanced bootstrap with $\omega \in \{0.25, 0.5, 0.75, 1\}$, or undersampling to 1,000 or 3,000 instances per class. Random trials cover a joint space more efficiently than a grid [1], and successive halving spends the budget on the promising configurations. The first stage drew 40 random configurations for each of the four families of class treatment and evaluated each with 100 trees on two folds. The second stage evaluated the four best configurations of each family and seven forced configurations, the optima of the sweeps and of the first results, with 200 trees on five folds and three runs, and the best of each family became the tuned forest. The number of trees was left to the size experiment, because the forest converges as the number of trees grows.

The first boosting grid spanned the depths three to 18 with 200 rounds. The learning rates span a roughly logarithmic range up to one, the rate without shrinkage, and the class-weight powers match the sweeps of the forest. The optimum lay at the edge, so a second grid added the depths 24 and 32, lower learning rates and 300 rounds. The vote averages the class distributions of the members with equal weights. The stacked ensemble trains a class-weighted meta-forest of 100 trees on the out-of-fold class distributions of the four members. Five grouped folds inside the training part produce the distributions, so no member predicts a row with the help of an identical vector.

D. Performance Measures

The precision u_j, the recall e_j and the F1 score f_j of class j follow from the confusion matrix, with zero for a zero denominator. The main measure is the macro-averaged F1 score $\bar{f}$, the mean of f_j over the m classes, because accuracy follows the frequent classes and hides failures on the rare classes [23]. Brodersen *et al.* showed that the accuracy is optimistic for a biased classifier on imbalanced data and proposed the balanced accuracy, the mean recall over the classes [6]. The study also reports the balanced accuracy.

The strength $\mathcal{S}$ is the mean accuracy of the members. The disagreement Θ follows equation (1). For two members, let N^{11}, N^{00}, N^{10} and N^{01} count the instances where both, neither, only the first or only the second member is correct. The Yule statistic of the pair is $(N^{11}N^{00} - N^{01}N^{10})/(N^{11}N^{00} + N^{01}N^{10})$, and $\mathcal{Q}$ is the mean of the statistic over all pairs, so a high $\mathcal{Q}$ means that the members tend to be right and wrong together. Given the doubts of Kuncheva and Whitaker, the measures serve as descriptions only. The decomposition follows the definitions of Breiman [3] over R training sets. The bias β is the share of test instances whose main prediction is wrong, and the error $\mathcal{E}$ is the mean error over the instances and the training sets. The variance $\mathcal{V}$ is the mean share of predictions that differ from the main prediction, whether the difference helps or hurts. The net variance $\mathcal{N} = \mathcal{E} - \beta$ is a different quantity: the harmful differences minus the helpful ones. A model can therefore have a large $\mathcal{V}$ and a net variance near zero, and the gains below are stated in terms of $\mathcal{N}$. The bias β_o replaces each label by the most frequent label of the same feature vector in D. The replacement uses the test labels, so β_o is descriptive only.

E. Statistical Tests

Let $\xi_{i,h}$ denote the score of model h on partition i, for $i = 1, \ldots, z$ with $z = 15$. The paired difference of models h and h' is $\psi_i = \xi_{i,h} - \xi_{i,h'}$, with mean $\bar{\psi}$ and sample standard deviation v. Rainio *et al.* recommended the Wilcoxon signed-rank test for the matched scores of two models [20]. The study reported the two-sided exact test with the paired effect size $\gamma = \bar{\psi}/v$. The overlap of the partitions inflates the effect sizes, so the effect sizes only order the differences. The overlap also makes the p-values optimistic, so the corrected resampled t-test of Nadeau and Bengio inflates the variance of the mean difference [17]. Equation (9) gives the statistic τ, with $\kappa = 1/4$

the ratio of the test size to the training size and $z - 1$ degrees of freedom.

$$\tau = \frac{\bar{\psi}}{\sqrt{(1/z + \kappa)\, v^2}} \tag{9}$$

Each family of tests in one table was adjusted with the step-down procedure of Holm [12]. Two models counted as different only when the adjusted corrected p-value was below 0.05. A mean difference below 0.005 in the macro-averaged F1 score counted as equivalence, and practical sizes are given as multiples of the margin or as correctly classified flows per thousand. Statistical significance and practical size are separate verdicts, and a significant difference need not exceed a few margins. The score of a forest on a partition is the mean over the runs of the forest. The units of the tuning experiments are not independent, so tuning results carry mean differences and effect sizes but no p-values. Scores on the generalisation sets carry bootstrap intervals over the test instances with 1,000 resamples, and two models are compared with the paired bootstrap of the difference.

F. Changes to the Initial Design

The initial design followed the one-factor sweeps with one random search that selected a single forest without and a single forest with a class treatment. That search drew only six configurations without a class treatment, so the joint search with four families replaced that search. The edge optimum of the first boosting grid led to the second grid. Before the final comparison, the numbers of runs rose from three to five per comparison partition, from ten to 30 on G and from one to three in tuning, and the decomposition grew from one to five folds. The initial design called the forest with class weights balanced, and the final report names the forests by the mechanism of the class treatment.

G. Summary

Nine questions guided the experiments on two protocols. The joint search tuned the forests, and the extended grids tuned boosting. The decomposition and the boosting weight explain the mechanisms, and the dissimilar ensembles and the grouped protocol test the argument for similar members. The next section reports the results.

V. Research Results

The forests raised the score of the tree mainly through a lower net variance, and a mild class treatment protected the rare classes. The headline scores come first, so that every tuning result is read against the final comparison. The tuning experiments, the decomposition and the out-of-bag error follow. The comparisons under both protocols, the dissimilar ensembles and a discussion close the Results.

A. Headline Scores

Table III gives the scores of all eight models under both protocols, as means and standard deviations over the 15 partitions and as point scores on the generalisation sets. The class-sensitive forests had the highest macro-averaged F1 score on every evaluation set, and the plain forest had the highest accuracy. The balanced tree had the highest balanced accuracy, at the lowest accuracy of all models. All four ensembles scored higher than the tree and the weighted nearest-neighbour model in every column. In practical terms, the plain forest classified 21 more flows per thousand correctly than the tree on the compatible partitions. The gain of the weighted forest in macro-averaged F1 score is 11 times the equivalence margin. The tuning scores below are means and standard deviations over five folds and two runs unless stated otherwise.

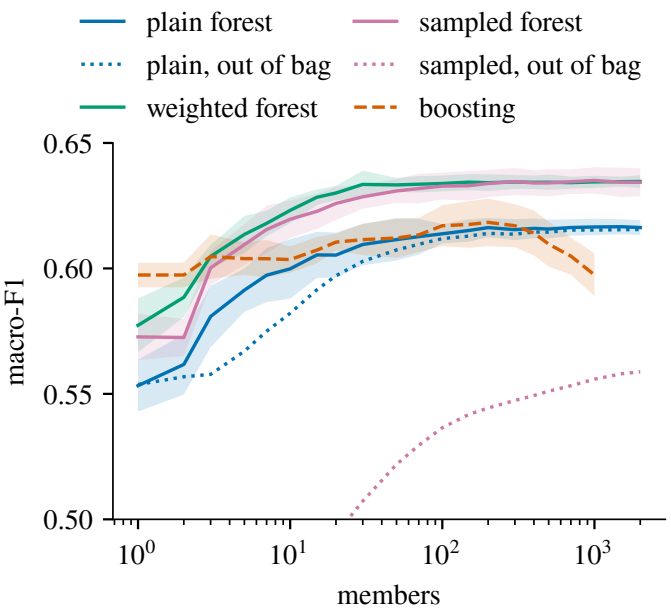


Fig. 1: Effect of the number of members

B. Encoding, Size and Structure

The one-hot encoding scored 0.602 ± 0.010 against 0.595 ± 0.010 for the ordinal rank, whose thresholds group unrelated categories, so all later experiments kept the one-hot encoding.

The forests reached a plateau within 50 trees and did not decline up to 2,000 trees, as Figure 1 shows, in line with the convergence that Breiman proved [4]. The returns diminished at 50 trees for the plain forest, 20 for the weighted forest and 50 for the sampled forest, before the expected 100 trees. The 500 trees of the comparison therefore lie on the plateau. Boosting, unlike the forests, peaked and then declined. The tuned ensemble reached 0.618 ± 0.009 after 200 rounds and fell to 0.597 after 1000 rounds, while the training error stayed near 0.090 and the disagreement rose from 0.249 to 0.338. The decline agrees with the expected effect of label noise, although the curve alone does not show the cause. Contrary to the expectation, the optimum came later than the plateau of the forests.

Figure 2 shows the effect of the number φ of features tried per split. The score rose from 0.528 at one feature to 0.602 at the default square root, peaked at 0.616 with 16 features and fell to 0.609 for plain bagging. The strength of a tree rose only

TABLE III: Scores of the models under the compatible and the grouped protocol

	15 partitions, mean ± standard deviation			Generalisation set		
Model	Accuracy	Balanced accuracy	Macro-F1	Accuracy	Balanced accuracy	Macro-F1 [interval]
Compatible protocol: partitions of D and the set G						
k-nearest neighbours	0.799 ± 0.001	0.506 ± 0.009	0.523 ± 0.010	0.802	0.502	0.516 [0.500, 0.531]
k-nearest neighbours, weighted	0.800 ± 0.002	0.533 ± 0.008	0.528 ± 0.005	0.803	0.530	0.529 [0.512, 0.545]
tree	0.812 ± 0.002	0.566 ± 0.012	0.580 ± 0.010	0.816	0.552	0.572 [0.554, 0.587]
tree, balanced	0.744 ± 0.007	**0.666 ± 0.013**	0.569 ± 0.006	0.747	**0.666**	0.570 [0.556, 0.583]
forest, plain	**0.832 ± 0.001**	0.595 ± 0.009	0.615 ± 0.007	**0.835**	0.576	0.602 [0.584, 0.618]
forest, weighted	0.823 ± 0.001	0.654 ± 0.011	**0.634 ± 0.005**	0.824	0.658	0.633 [0.620, 0.645]
forest, sampled	0.823 ± 0.001	0.647 ± 0.010	0.632 ± 0.007	0.826	0.655	**0.637** [0.623, 0.648]
boosting	0.819 ± 0.001	0.612 ± 0.010	0.615 ± 0.008	0.823	0.614	0.613 [0.597, 0.628]
Grouped protocol: grouped partitions of D_g and the set G_g						
k-nearest neighbours	0.800 ± 0.004	0.493 ± 0.010	0.504 ± 0.015	0.778	0.491	0.489 [0.474, 0.504]
k-nearest neighbours, weighted	0.803 ± 0.002	0.537 ± 0.008	0.528 ± 0.007	0.784	0.538	0.523 [0.507, 0.538]
tree	0.815 ± 0.002	0.571 ± 0.009	0.582 ± 0.006	0.798	0.576	0.575 [0.559, 0.589]
tree, balanced	0.755 ± 0.005	**0.686 ± 0.006**	0.585 ± 0.009	0.742	**0.681**	0.558 [0.544, 0.570]
forest, plain	**0.838 ± 0.001**	0.595 ± 0.008	0.611 ± 0.007	**0.821**	0.597	0.601 [0.585, 0.616]
forest, weighted	0.825 ± 0.002	0.669 ± 0.007	**0.643 ± 0.006**	0.804	0.654	0.616 [0.600, 0.629]
forest, sampled	0.827 ± 0.002	0.666 ± 0.007	0.641 ± 0.007	0.807	0.657	**0.626** [0.611, 0.640]
boosting	0.822 ± 0.002	0.625 ± 0.011	0.619 ± 0.007	0.802	0.599	0.590 [0.574, 0.606]

from 0.775 to 0.799, whereas the disagreement fell from 0.197 to 0.163 and the Yule statistic $\mathcal{Q}$ rose from 0.890 to 0.915, so the trees erred more often together. A trade-off between strength and diversity therefore existed, but the strength varied little. The diversity moved monotonically with φ while the score peaked in the middle of the range, in line with the doubt that Kuncheva and Whitaker raised [13]. The default square root was not the optimum.

Deep trees with small leaves gave the highest score, as expected for averaging. The score grew from 0.250 at depth three to 0.601 without a depth limit, and fell to 0.493 at a leaf size of 50 while the accuracy changed little. The loss lay in the rare classes: the recall of Worms fell from 0.434 at a leaf size of one to 0.000 at 25. A large leaf stops the trees from splitting off the small pockets of a rare class, even though the strength of a single tree rose with the leaf size.

C. Class Treatment and Tuned Ensembles

A mild balanced bootstrap gave the largest gain at a small cost in accuracy, as Table IV shows. The balanced bootstrap with the exponent 0.5 raised the macro-averaged F1 score by 0.030, with a paired effect size of 3.8, and the recall of Worms from 0.446 to 0.690. Class weights with the power 0.5 gained 0.014. Stronger treatments lowered the score again, because the precision of the rare classes fell faster than the recall rose. Undersampling to the smallest class gave the highest balanced accuracy but the lowest score. Combining class weights with the balanced bootstrap gave 0.594, below either treatment alone, so the two treatments overlap. The recall of Worms varied by 0.139 between units for the plain forest, so only large differences on the class are reliable.

The joint search confirmed the sweeps. In the second stage, the three families with a class treatment were equivalent: the weighted forest reached 0.634 ± 0.003, the sampled forest

TABLE IV: Class treatment of the forest

					Recall e_j		
Sampler	α	Macro-F1	Acc.	Bal. acc.	Backdoor	Analysis	Worms
standard	0	0.601 ± .009	0.832	0.574	0.092	0.112	0.446
standard	0.5	0.614 ± .007	0.825	0.603	0.124	0.123	0.546
standard	1	0.592 ± .008	0.800	0.588	0.189	0.152	0.575
$\omega = 0.5$	0	0.631 ± .003	0.823	0.640	0.168	0.140	0.690
$\omega = 1$	0	0.588 ± .005	0.774	0.617	0.223	0.166	0.712
under 1,000	0	0.531 ± .006	0.728	0.650	0.229	0.276	0.889
under Worms	0	0.481 ± .003	0.709	0.661	0.326	0.309	0.946

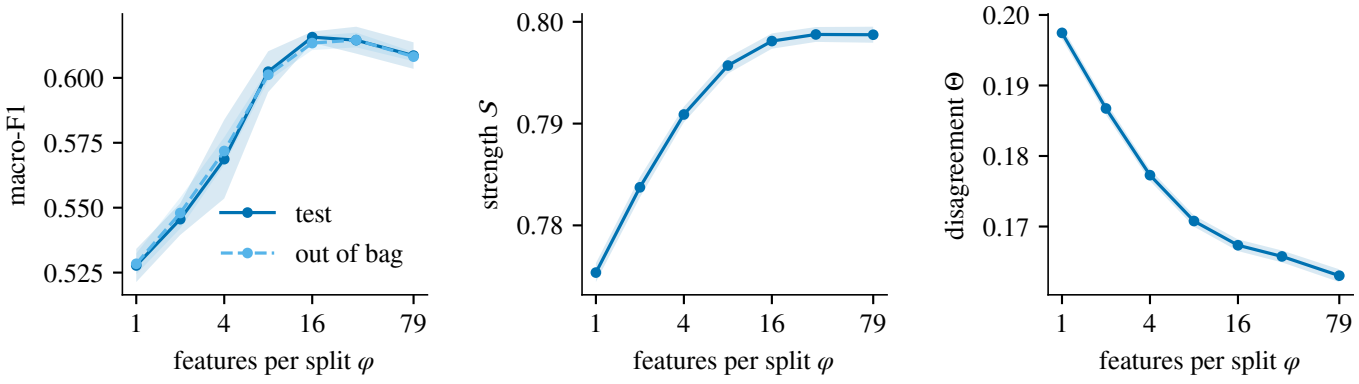
0.634 ± 0.005 and the forest with both treatments 0.634 ± 0.007. The data therefore do not separate the class weights from the balanced bootstrap. The plain forest scored lower by 0.018. The weighted forest uses 16 features per split, depth 20, a leaf size of 3 and $\alpha = 0.5$. The sampled forest uses 16 features, fully grown trees and $\omega = 0.5$, and the plain forest 24 features.

Boosting needed deep weak learners. The best grid score rose from 0.483 at depth three to 0.618 at depth 24 and fell to 0.612 at depth 32. The tuned ensemble uses depth 24, the learning rate 0.1, the class-weight power 0.5 and 200 rounds, and scored 0.618 ± 0.009. Even deep learners stayed below the weighted forest.

D. Bias, Variance and Boosting Weight

Table V gives the decomposition over the five tuning folds, with 20 training sets of 82,456 instances per fold. Differences above 0.01 are stable over the folds.

The plain forest lowered the error of the tree by 0.025: the net variance $\mathcal{N}$ fell by 0.020 and the bias by 0.005. Within the operational definitions of Section IV-D, about four fifths of the gain came from the reduction of the net variance, as expected. The variance $\mathcal{V}$ fell further, from 0.111 to 0.065,



(a) Score (b) Strength $\mathcal{S}$ (c) Disagreement Θ

Fig. 2: Effect of the number of features per split φ on the forest

TABLE V: Bias and variance

Model	$\mathcal{E}$	β	β_o	$\mathcal{N}$	$\mathcal{V}$	Macro-F1
tree	0.200	0.176	0.117	+0.024	0.111	0.554 ± 0.001
tree, balanced	0.259	0.249	0.250	+0.010	0.114	0.551 ± 0.004
bagging	0.178	0.173	0.112	+0.005	0.068	0.588 ± 0.004
forest, plain	0.175	0.171	0.110	+0.004	0.065	0.593 ± 0.005
forest, weighted	0.184	0.180	0.154	+0.004	0.059	0.611 ± 0.005
forest, sampled	0.192	0.184	0.152	+0.008	0.084	0.598 ± 0.004
boosting	0.196	0.182	0.151	+0.014	0.095	0.578 ± 0.005

because $\mathcal{V}$ also counts the helpful differences. Plain bagging reached nearly the same error, so the random feature subsets added little at the training-set size of the decomposition. The class treatment moved errors from the rare classes to the frequent classes. The weighted forest kept the net variance of the plain forest, while the bias on Worms fell from 0.461 to 0.288 and on denial of service (DoS) from 0.724 to 0.360. Per class, the decomposition locates the remaining error. On Backdoor and Analysis, the bias stayed at 0.918 and 0.916 for the plain forest, against 0.906 and 0.904 for the tree, with net variances of -0.002 and -0.002, so averaging had no variance left to remove on those classes. On Worms, the plain forest lowered the net variance from +0.085 to +0.046 at a nearly unchanged bias. Even the weighted forest kept a bias of 0.856 on Backdoor and 0.885 on Analysis. The sampled forest had a higher variance, because a balanced bootstrap repeats the few rare instances many times. Boosting shares the class weights of the weighted forest, so the weighted forest is the reference for boosting. Contrary to both expectations, boosting reduced the bias no further than the weighted forest and kept more variance than the forests, although less than the tree. The decomposition covered only the tuned deep learners.

Boosting placed nearly all of the instance weight on the conflicting vectors within the first rounds. Rows of conflicting vectors make up 0.155 of the training rows, yet their share of the weight rose from 0.274 after the first round to 0.667 after ten rounds and 0.997 after 100 rounds. The conflicting vectors hold every row that no deterministic classifier can fit. The concentration and the decline after the peak agree with

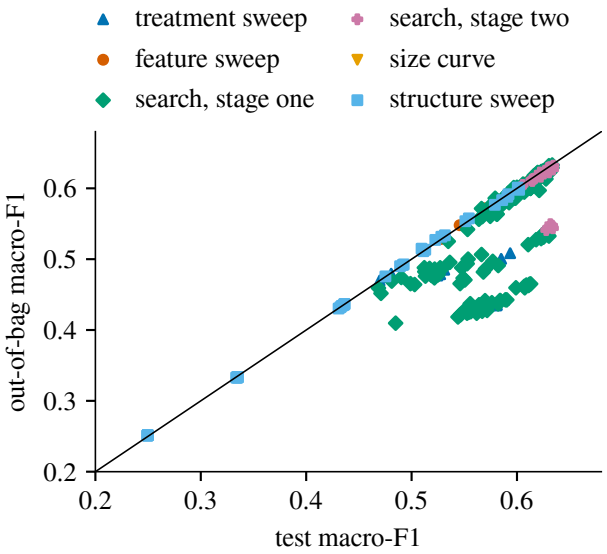


Fig. 3: Out-of-bag against test score of all configurations

the conflicting rows driving the decline, but deep learners and class weights also shape the rounds.

E. Out-of-Bag Error as a Validation Signal

The out-of-bag score ranked distant configurations well and close configurations poorly. Figure 3 plots the out-of-bag against the test score of 242 forest configurations. The rank correlation was 0.783 over all configurations and 0.715 above a test score of 0.5, but fell to 0.308 within the close configurations of the second search stage. The out-of-bag accuracy differed from the test accuracy by 0.0003 on average.

The out-of-bag macro-averaged F1 score lay below the test score by 0.034 on average, most of all for the class-balanced samplers in the cluster below the diagonal. A balanced bootstrap draws each instance of a rare class many times, so the instance is out of bag for few trees. Within the four families, the out-of-bag choice matched the cross-validated choice in three families, with a largest loss of 0.002.

F. Comparison under the Compatible Protocol

Every ensemble scored higher than the tree under the compatible protocol, as Table III shows. The weighted forest scored higher than the tree by 0.053 in all 15 partitions, with $p < 0.001$ for the adjusted corrected t-test and $p = 0.002$ for the adjusted Wilcoxon test, and raised the balanced accuracy by 0.087. The plain forest scored higher than the tree by 0.035 ($p < 0.001$) and higher than the weighted nearest-neighbour model by 0.088. The sampled forest and boosting also scored higher than the tree, by 0.052 ($p < 0.001$) and 0.035 ($p < 0.001$), so all four gains over the tree were significant after the Holm adjustment. The gains were also practically large, between seven and 11 times the equivalence margin. The two class-sensitive forests did not differ ($p = 1.000$), whereas boosting scored lower than the weighted forest by 0.019 ($p = 0.001$). Boosting and the plain forest differed by only 0.001 ($p = 1.000$), a difference that is neither significant nor practically relevant. The gap to the weighted forest lies in the rare classes, as the accuracies differed by only three flows per thousand. The class treatment cost the plain forest 10 correctly classified flows per thousand. On G, the paired bootstrap interval of the weighted forest against the tree, $[+0.047, +0.077]$, excludes zero. The intervals of the sampled against the weighted forest and of boosting against the plain forest include zero.

The gain came from the test instances without a training twin. On average, 55.9 per cent of the test instances had no twin, 28.9 per cent had twins of one label and 15.2 per cent had twins with conflicting labels. Every model classified the consistent twins almost perfectly and the conflicting twins poorly, the tree with accuracies of 0.984 and 0.371. Without a twin, the accuracy rose from 0.842 for the tree to 0.877 for the plain forest, the expected answer to the comparison question.

The class treatment raised the F1 score on G of Worms from 0.488 for the plain forest to 0.720 for the weighted forest, and of DoS from 0.302 to 0.475. Worms has only 35 instances in G, and every model stayed below 0.2 on Backdoor and Analysis. Figure 4 shows the confusion of the weighted forest. More than half of the Backdoor and Analysis flows were predicted as DoS, and DoS and Exploits exchanged a quarter or more of the flows of each class. In D, 85 per cent of the Backdoor flows and 76 per cent of the Analysis flows sit in vectors with conflicting labels, against 1 per cent of the normal flows.

A split of G tests the role of the conflicting vectors. On the 84.3 per cent of the rows outside conflicting vectors, the weighted forest predicted 5 per cent of the Backdoor flows and 3 per cent of the Analysis flows as DoS, against 65 and 68 per cent inside conflicting vectors. The conflicting labels are therefore the main source of that confusion. The confusion of DoS with Exploits persisted on the clean rows, at 51 per cent, so the two classes overlap in the feature space beyond the identical vectors.

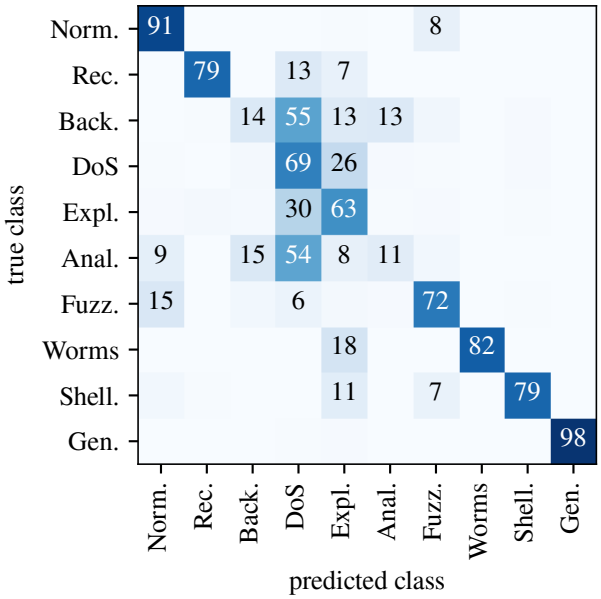


Fig. 4: Row-normalised confusion matrix of the weighted forest on G in per cent

G. Grouped Protocol and Dissimilar Ensembles

The grouped scores stayed close to the compatible scores, although no test vector had a twin. The plain nearest-neighbour model fell from 0.523 to 0.504, as expected for a memorising model. The accuracy was slightly higher on the grouped partitions, for a cause that is not established. The Kendall rank correlation of the mean scores of the eight models between the protocols was 1.00 for the accuracy, 1.00 for the balanced accuracy and 0.86 for the macro-averaged F1 score. Two pairs changed places: the two single trees, and the plain forest and boosting. Each of those pairs lay within the equivalence margin under one of the protocols. The weighted forest stayed above the weighted nearest-neighbour model by 0.115 ($p < 0.001$), so the ranking of the ensembles above the single models held.

The gains over the tree did not shrink on the grouped partitions. The weighted forest scored higher than the tree by 0.060 ($p < 0.001$), against 0.053 under the compatible protocol. The sampled forest gained 0.059 against 0.052, and boosting 0.037 against 0.035. Only the gain of the plain forest fell, from 0.035 to 0.029. All four gains stayed significant, each with $p < 0.001$. The unchanged gains agree with Section V-F, where the gain came from the test instances without a twin, the only kind of test instance in a grouped partition.

The single grouped split G_g gave a different picture. On G_g, every model had a lower accuracy than on G, by five to 24 flows per thousand. The gain over the tree was smaller for every ensemble. The gain of the weighted forest fell from $+0.062$ to $+0.041$, that of the plain forest from $+0.030$ to $+0.026$, that of the sampled forest from $+0.065$ to $+0.051$ and that of boosting from $+0.041$ to $+0.016$. The two generalisation sets hold different rows, and the 15 grouped

partitions show no such shrinkage. The smaller gains on G_g may therefore be specific to that one split, and the comparison is not an isolated causal estimate of the effect of the twins.

No dissimilar combination scored higher than the best forest. Under the compatible protocol, the vote of all four members scored 0.607, lower than the weighted forest by 0.027 ($p < 0.001$), and the vote of the two forests scored 0.622. The stacked ensemble scored 0.630, within 0.004 of the weighted forest, and lay below the weighted forest by 0.011 ($p = 0.001$) under the grouped protocol. Leaving the tree out of the vote raised the score by 0.007, and leaving the nearest-neighbour model out lowered the score by 0.006. The nearest-neighbour model thus adds a little diversity, but the equal weights let the weaker members pull the vote below the strongest member. As expected, no combination improved on the best forest by more than 0.005.

H. Discussion

Three findings cut across the questions. First, averaging saturates early and removes mainly variance. Much of the remaining bias sits in the attack categories whose flows share conflicting vectors, and neither more trees nor boosting reduced that part. Second, the rare classes respond to the class treatment and to deep trees with small leaves, not to the number of trees. A mild treatment raised the score by 0.018 in the joint search. Third, the repeated vectors did not change the order of the models, and the evidence on the size of the gain is mixed. The gains held on the 15 grouped partitions, as the twin analysis predicts, but shrank on the single set G_g. The two generalisation sets differ in their rows, so the smaller gains on G_g are consistent with an effect of the twins or with a property of that one split, not an inference of either.

The evidence on boosting has three levels. The observation is that boosting peaked and then declined, and that nearly all of the instance weight moved to the conflicting rows. The interpretation is that boosting fits label noise, in line with the finding of Breiman that forests are more robust to noise. A causal test would refit boosting without the minority labels of the conflicting vectors. The study did not run that test, so label noise remains the most plausible explanation and not a proven cause.

Five limitations qualify the findings. First, the tuned configurations came from the development set that the comparison partitions reuse, although the baselines share the effect. Second, the 15 partitions overlap, so the tests remain optimistic after the correction, and the conclusions also rest on effect sizes and the generalisation sets. Third, Worms has about 28 test instances per fold and 35 in G, so the scores of Worms are noisy. Fourth, the study did not build gradient boosting. Fifth, published results on UNSW-NB15 use other partitions, class sets and preprocessing, so the study does not compare the present scores with the published scores.

I. Summary

The forests converged after 20 to 50 trees and gained mainly through a lower net variance, and a mild class treatment helped the rare classes. The ranking and the gains over the tree held on the grouped partitions, and only the grouped generalisation set showed smaller gains. Boosting scored lower than the weighted forest and placed nearly all of the instance weight on conflicting rows. No dissimilar ensemble scored higher than the best forest.

VI. Conclusion

The study measured how much ensembles of classification trees gain on imbalanced attack categories with conflicting vectors, and why. The Conclusion answers the research questions and states the limitations and future work.

A. Answers to the Research Questions

The study built a random forest and the stagewise additive modelling using a multi-class exponential loss (SAMME) variant of adaptive boosting from scratch. Under the compatible protocol, the macro-averaged F1 score rose from 0.580 for the tree to between 0.615 and 0.634 for the ensembles. All four gains over the tree were significant after the Holm adjustment under both protocols.

The returns of the forests diminished after 20 to 50 trees without overfitting, whereas boosting peaked at 200 rounds and then degraded. As the number of features per split grew, the strength of a tree varied little while the diversity fell, and the optimum of 16 to 24 features lay above the default. Fully grown trees with small leaves were best. A mild class treatment raised the score, and class weights and the balanced bootstrap were equivalent on the partitions and on G. About four fifths of the gain of the plain forest came from a reduction of the net variance. The boosting weight moved to the rows of conflicting vectors, a shift that agrees with sensitivity to label noise but does not prove such sensitivity. The out-of-bag error estimated the accuracy closely but separated close configurations poorly. The ensembles scored higher than both models of the second assignment, and the gain came from the test instances without a twin. Under the grouped protocol, the ranking held and the gains over the tree did not shrink on the partitions, apart from a small decrease for the plain forest. The gains were smaller only on the single set G_g, whose rows differ from those of G. No dissimilar ensemble improved on the best forest.

B. Limitations and Future Work

The overlap of the partitions, the reuse of the development set and the small Worms class limit the precision of the conclusions. Boosting without the minority labels of the conflicting vectors would test the cause of the decline of boosting, and grouped and ungrouped splits of one pool of rows would isolate the effect of the twins. Gradient boosting and tuning under the grouped protocol are further directions.

References

[1] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," *Journal of Machine Learning Research*, vol. 13, pp. 281–305, 2012.

[2] L. Breiman, "Bagging predictors," *Machine Learning*, vol. 24, no. 2, pp. 123–140, 1996.

[3] ——, "Arcing classifier (with discussion and a rejoinder by the author)," *The Annals of Statistics*, vol. 26, no. 3, pp. 801–849, 1998.
[4] ——, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
[5] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*. Chapman and Hall/CRC, 2017.
[6] K. H. Brodersen, C. S. Ong, K. E. Stephan, and J. M. Buhmann, "The balanced accuracy and its posterior distribution," in *Proceedings of the 2010 20th International Conference on Pattern Recognition*, 2010, pp. 3121–3124.
[7] T. Cover and P. Hart, "Nearest neighbor pattern classification," *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967.
[8] Y. Freund and R. E. Schapire, "A decision-theoretic generalization of on-line learning and an application to boosting," *Journal of Computer and System Sciences*, vol. 55, no. 1, pp. 119–139, 1997.
[9] J. H. Friedman, "Greedy function approximation: a gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.
[10] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.
[11] T. Hastie, S. Rosset, J. Zhu, and H. Zou, "Multi-class AdaBoost," *Statistics and Its Interface*, vol. 2, no. 3, pp. 349–360, 2009.
[12] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.
[13] L. I. Kuncheva and C. J. Whitaker, "Measures of diversity in classifier ensembles and their relationship with the ensemble accuracy," *Machine Learning*, vol. 51, no. 2, pp. 181–207, 2003.
[14] S. K. Lam, A. Pitrou, and S. Seibert, "Numba: a LLVM-based Python JIT compiler," in *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*. Association for Computing Machinery, 2015, pp. 1–6.
[15] X.-Y. Liu, J. Wu, and Z.-H. Zhou, "Exploratory undersampling for class-imbalance learning," *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, vol. 39, no. 2, pp. 539–550, 2009.
[16] N. Moustafa and J. Slay, "UNSW-NB15: a comprehensive data set for network intrusion detection systems," in *Proceedings of the 2015 Military Communications and Information Systems Conference*, 2015, pp. 1–6.
[17] C. Nadeau and Y. Bengio, "Inference for the generalization error," *Machine Learning*, vol. 52, no. 3, pp. 239–281, 2003.
[18] P. Probst, M. N. Wright, and A.-L. Boulesteix, "Hyperparameters and tuning strategies for random forest," *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 9, no. 3, p. e1301, 2019.
[19] J. R. Quinlan, "Induction of decision trees," *Machine Learning*, vol. 1, no. 1, pp. 81–106, 1986.
[20] O. Rainio, J. Teuho, and R. Klén, "Evaluation metrics and statistical tests for machine learning," *Scientific Reports*, vol. 14, no. 1, p. 6086, 2024.
[21] C. Strobl, A.-L. Boulesteix, A. Zeileis, and T. Hothorn, "Bias in random forest variable importance measures: illustrations, sources and a solution," *BMC Bioinformatics*, vol. 8, no. 1, p. 25, 2007.
[22] K. M. Ting, "An instance-weighting method to induce cost-sensitive trees," *IEEE Transactions on Knowledge and Data Engineering*, vol. 14, no. 3, pp. 659–665, 2002.
[23] J. P. van Zyl and A. P. Engelbrecht, "Analysis of classification metric behaviour under class imbalance," *Egyptian Informatics Journal*, vol. 31, p. 100711, 2025.
[24] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau *et al.*, "SciPy 1.0: fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, no. 3, pp. 261–272, 2020.
[25] D. H. Wolpert, "Stacked generalization," *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.

Appendix A
Acronyms

The report uses the two acronyms listed below. UNSW-NB15, the name of the data set, and F1, the name of a score, are names rather than acronyms.

DoS	denial of service, an attack category
SAMME	stagewise additive modelling using a multi-class exponential loss

Appendix B
Symbols

The symbols follow in alphabetical order, Latin letters first and Greek letters second.

$a_{i,j}$	votes for class j on instance i
B, c	number of members, member index
b_j	class weight of class j
D, G	development and generalisation set
D_g, G_g	grouped development and generalisation set
$\mathcal{E}$	error over training sets
e_j	recall of class j
f_j, $\bar{f}$	F1 score of class j, macro-averaged F1
$g(t)$, $H(t)$	Gini index and entropy
i, j	instance index, class index
h, h'	model indices
k	number of neighbours
m	number of classes
$\mathcal{N}$	net variance
N^{11}, N^{00}	instances where both or neither member is correct
N^{10}, N^{01}	instances where only one member is correct
n, n_j	training instances, of class j
$P_{c,j}(\mathbf{x})$	leaf proportion of class j
$p_{c,i}$	boosting weight of instance i
$\mathcal{Q}$	mean Yule statistic of member pairs
q	undersampling size per class
R	number of training sets
$\mathcal{S}$	strength of the members
s	candidate split
$\mathcal{T}$	evaluated set of instances
t, t_L, t_R	node and children
u_j	precision of class j
$\mathcal{V}$	variance
v	deviation of paired differences
w_i	vote weight of neighbour i
$\mathbf{x}$, $\mathbf{x}_i$	feature vector, of instance i
y_i	label of instance i
$\hat{y}(\mathbf{x})$	predicted class
$\tilde{y}(\mathbf{x}_i)$	out-of-bag prediction
z	number of partitions
α	class-weight power
β, β_{o}	bias against label, against best label
γ	paired effect size
$\Delta(s,t)$	impurity decrease
η_L, η_R	shares of the node weight
Θ	disagreement of members
ι	indicator function
κ	ratio of test to training size
$\mu_c(\mathbf{x})$	class predicted by member c
ν_c	model weight of member c
$\pi_{t,j}$	class proportion at node t
ϱ	learning rate
σ_c	weighted error of member c
τ	corrected t statistic
ϕ	impurity measure

φ	features tried per split
$\chi_{c,i}$	multiplicity of instance i
ψ_i, $\bar{\psi}$	paired difference and mean
$\xi_{i,h}$	score of model h on partition i
ω	class-balance exponent
$\Omega_k(\mathbf{x})$	indices of the neighbours